

Politecnico di Torino

CORSO DI LAUREA MAGISTRALE IN INGEGNERIA INFORMATICA

MASTER OF SCIENCE IN COMPUTER ENGINEERING

Large Language Models for Software Engineering

Official Examination – Summer Session: June 19, 2026

Course Code:	01URXOV	Academic Year:	2025/2026
Exam Date:	June 19, 2026	Time Window:	09:10 – 10:24 (1h 14m)
Total Questions:	11 Questions	Max Evaluation:	18.00 Points
Format:	Closed-book, digital delivery	Platform:	Moodle / Polito Exam Platform

Structure of the Examination Document:

- **Part I: Exam Questions** contains the complete verbatim text, options, point distribution, and penalty rules for all 11 questions exactly as presented during the exam session. Zero omission of questions, choices, or instructions.
- **Part II: Comprehensive Solutions & In-Depth Analysis** provides complete, publication-grade academic solutions: rigorous mathematical derivations, architectural diagrams, prompt engineering templates, error analysis, and theoretical references grounded in state-of-the-art LLM literature.

1 Part I: Exam Questions

Question 1

[1.50 Points | No Penalty]

You are given the following sequence of characters:

c c b a a a d a a c c c

Initially, each character is a separate token.

Apply Byte-Pair Encoding (BPE) to produce 3 additional tokens. What are the generated tokens?

Write each token, in the correct order, in the following slots. Separate the merged tokens with a comma (and no spaces).

- *Example 1:* If the merged tokens are **aa** and **b**, write: **aa,b**
- *Example 2:* If the merged tokens are **x** and **y**, write: **x,y**

- **1st merge:** _____
- **2nd merge:** _____
- **3rd merge:** _____

Question 2

[3.00 Points | No Penalty]

Consider a scenario where a software team wants to use LLaMA-based agents to help improve a pull request.

You are given:

- **FEATURE_SPEC**: a natural language description of the feature
- **PR_CODE**: the code submitted in the pull request
- **TEST_RESULTS**: the results of running tests on the code, including passed and failed tests

Design a simple agentic workflow that uses LLaMA-based agents to:

1. Understand the feature requirements
2. Improve the code in the pull request
3. Use test feedback to refine the code if tests fail

For each agent, briefly specify:

- Its role
- Its system prompt
- Its input
- Its output

Note: You do not need to write code or actual LLaMA API calls. Focus only on the agents, their prompts, and how information flows between them.

Question 3

[1.00 Point | Penalty: -25% (-0.25 pt)]

What is “priming” in the context of prompt engineering?

- a. Providing initial input to guide the model’s response
- b. Resetting the model before every new prompt
- c. Using reinforcement learning to fine-tune the model
- d. Pre-training the model with additional data
- e. Adding noise to the input to test model robustness

Question 4

[1.50 Points | Penalty: -25% (-0.38 pt)]

The following reward formulation can be used when fine-tuning a model on human feedback:

$$R(x, y) = r(x, y) - \beta \log \left(\frac{\pi(y | x)}{\pi_0(y | x)} \right)$$

Where $r(x, y)$ is the output of a Reward Model, π and π_0 are the new and the original policies, respectively (you can see them as the probabilities assigned by the new and original models).

What is the purpose of the term $\log \left(\frac{\pi(y|x)}{\pi_0(y|x)} \right)$?

- a. it helps make the model more deterministic
- b. it improves the model’s capability of following instructions
- c. it rewards the model when it generates an answer that humans will “like”

- d. none of the other answers is correct
- e. it prevents the model from collapsing into a model that always predicts a single token
- f. it increases the entropy of the output (i.e., it helps the model produce more “creative” answers)
- g. it prevents the new model from producing outputs that are too similar to the original one

Question 5**[1.50 Points | Penalty: -15% (-0.23 pt)]**

You are given an encoder-only model. What is the total number of parameters (token + positional + transformer layers + language model head)?

Note: assume that none of the layers uses a bias term. You can ignore layer normalization parameters.

- Vocabulary size $V = 29,000$
- Embedding / model dimension $d = 256$
- Positional embeddings: absolute, learned, for 512 positions
- Number of layers $L = 9$
- Number of attention heads $H = 16$
- FF-NN projection dimension: $d_{\text{ff}} = 32$
- Key, query size $d_k = 4$
- Value size $d_v = 64$

Choose the correct answer. Round the result to the closest million.

Answer: _____

Question 6**[1.50 Points | Penalty: -25% (-0.38 pt)]**

In a Transformer’s self-attention mechanism, what is the primary purpose of the matrices W_Q , W_K , and W_V ?

- a. They are used to transform the input token embeddings into vectors that are passed to the attention mechanism
- b. They are used to store the query, key, and value vectors learned during training for each token
- c. They are passed to the attention mechanism to compute attention scores between tokens
- d. None of the other answers is correct
- e. They are used to transform positional encodings into query, key, and value vectors before attention
- f. They are used to transform the output of the attention mechanism back into token embeddings
- g. They are used to combine the outputs of multiple attention heads into a single representation

Question 7

[1.50 Points | No Penalty]

You want to fine-tune a decoder-only transformer-based model using LoRA. The details of the model are reported below:

- Vocabulary size $V = 74,000$
- Embedding / model dimension $d = 4096$
- Positional embeddings: absolute, learned, for 1024 positions
- Number of layers $L = 20$
- Number of attention heads $H = 8$
- FF-NN projection dimension: 1024
- Key, query size $d_k = 128$
- Value size $d_v = 256$

You apply LoRA to the W_Q , W_K and W_V matrices. You want to fine-tune at most 20 M parameters (20,000,000). What is the highest possible low-rank dimensionality (r) to be used? Assume that each of the attention heads has separate matrices for W_Q , W_K and W_V (many implementations stack the matrices for all heads into a single matrix – for this exercise, assume that each matrix is separate from the others). Do not consider bias terms in the computations.

Answer: _____

Question 8

[3.00 Points | No Penalty]

You are evaluating an LLM that extracts functional requirements from a specification for an online food delivery platform. The LLM is given a paragraph of system requirements and produces a list of extracted candidate requirements.

Specification:

“Customers can search restaurants by cuisine and location, view restaurant menus, add dishes to a cart, and place orders for delivery. The system allows users to track the order status in real time and pay online using credit cards or digital wallets. Restaurants must be able to accept or reject incoming orders. Delivery drivers should receive notifications when a new delivery is assigned to them.”

From this, the ground truth requirements are:

- **G1:** Search restaurants by cuisine and location
- **G2:** View restaurant menus
- **G3:** Add dishes to a cart
- **G4:** Place delivery orders
- **G5:** Track order status in real time
- **G6:** Pay online using credit cards or digital wallets
- **G7:** Allow restaurants to accept or reject incoming orders
- **G8:** Notify delivery drivers when a delivery is assigned

Requirements extracted by the LLM:

- **L1:** Find restaurants based on cuisine
- **L2:** Browse available menus
- **L3:** Add food items to a shopping cart
- **L4:** Reserve a table at a restaurant
- **L5:** Place an order for home delivery
- **L6:** Track delivery progress live

- **L7:** Pay using PayPal only
- **L8:** Send alerts to drivers for assigned deliveries
- **L9:** Allow restaurants to cancel customer accounts

Identify the following:

- True Positives (TP)
- False Positives (FP)
- False Negatives (FN)

Compute:

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}, \quad \text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}, \quad \text{F1-Score} = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

You can write in free text your assumptions about the matching between ground truth requirements and LLM-extracted requirements.

Question 9

[1.00 Point | Penalty: -25% (-0.25 pt)]

Which of the following are benefits of a Retrieval-Augmented-Generation (RAG) approach to prompting? *(Select all that apply)*

- It is possible to reduce the amount of false information provided to the final user.
- It is possible to adopt self-critique mechanisms to refine the outputs of the agent.
- It is possible to prevent the obsolescence of answers by using up-to-date documents.
- It is possible to reduce the amount of hallucinations by providing a fixed structure for the agent's answers.
- It is possible to decompose a task in several smaller steps.

Question 10

[1.50 Points | No Penalty]

The BLEU- n score is computed as follows:

$$\text{BLEU-}n = \text{BP} \times \exp \left(\sum_{i=1}^n w_i \log p_i \right)$$

Where p_i is the precision for i -grams (i.e., the fraction of matching i -grams between reference and generated sequences, w.r.t. the number of i -grams in the generated sequence), with uniform weights $w_i = 1/n$.

BP is the brevity penalty, defined as:

$$\text{BP} = \begin{cases} 1 & \text{if } c > r \\ e^{1-r/c} & \text{if } c \leq r \end{cases}$$

Where c is the length of the generated sequence and r is the length of the reference sequence. You are given the following sequences, already divided into tokens:

Reference (R): <tok0> <tok8> <tok11> <tok5> <tok9> <tok4> <tok1> <tok2>
<tok7> <tok19>
Generated (G): <tok3> <tok17> <tok1> <tok2> <tok7> <tok19> <tok5>

Compute the corresponding BLEU-2 score. Write the answer in the box below, reporting at least 4 digits after the decimal point.

Answer: _____

Question 11**[1.00 Point | Penalty: -25% (-0.25 pt)]**

In agentic behavior, what changes compared to a traditional fixed workflow?

- a. The model must always use RAG
- b. The model decides its own control flow and when to stop
- c. The model follows a predetermined control flow
- d. The model can't use tools
- e. The model always does multiple-step tasks

2 Part II: Comprehensive Solutions & In-Depth Analysis

Solution to Question 1: Byte-Pair Encoding (BPE) Tokenization

Official Final Answers:

Merge Step	Merged Representation	Exact Format Required
1st merge:	Pair (a, a) merged into new token aa	a,a
2nd merge:	Pair (c, c) merged into new token cc	c,c
3rd merge:	Pair (aa, a) merged into new token aaa	aa,a

1. Theoretical Background & Principles of BPE

Byte-Pair Encoding (BPE) is a data-driven subword tokenization algorithm originally introduced for data compression (Gage, 1994) and adapted for neural machine translation and language modeling by Sennrich et al. (2016). BPE addresses the fundamental vocabulary tradeoff between word-level models (which suffer from out-of-vocabulary [OOV] words and massive embedding matrices) and character-level models (which produce excessively long sequences and suffer from degraded long-range dependency modeling).

The algorithm operates iteratively:

1. Initialize the base vocabulary V_0 with all unique elementary characters present in the corpus.
2. Treat every sequence in the corpus as a list of individual character tokens.
3. At iteration k , count the frequencies of all adjacent token pairs (t_i, t_{i+1}) .
4. Identify the most frequent adjacent pair (t_A, t_B) . In case of frequency ties, a consistent deterministic tie-breaking policy is followed (such as first occurrence or lexicographical order).
5. Form a new composite token $t_{\text{new}} = t_A \circ t_B$, add t_{new} to vocabulary $V_k = V_{k-1} \cup \{t_{\text{new}}\}$, and replace all non-overlapping occurrences of the contiguous pair (t_A, t_B) from left to right across the corpus.

2. Step-by-Step Mathematical Derivation and Trace

We are given the initial sequence of $N = 13$ single-character tokens:

$$S_0 = [c, c, b, a, a, a, d, a, a, a, c, c, c]$$

Base vocabulary: $V_0 = \{a, b, c, d\}$.

Iteration 1 (Derivation of 1st Merge): We scan S_0 from left to right and compute the frequency of every adjacent pair:

Pair	Occurrences / Indices (0-indexed)	Frequency
(c, c)	(0, 1), (10, 11), (11, 12)	3
(c, b)	(1, 2)	1
(b, a)	(2, 3)	1
(a, a)	(3, 4), (4, 5), (7, 8), (8, 9)	4
(a, d)	(5, 6)	1
(d, a)	(6, 7)	1
(a, c)	(9, 10)	1

The most frequent adjacent pair is **(a, a)** with a frequency of 4. Under standard left-to-right greedy non-overlapping replacement:

- The first contiguous run **a a a** at indices 3..5 becomes **aa a**.
- The second contiguous run **a a a** at indices 7..9 becomes **aa a**.

Therefore, the **1st merge** is between **a** and **a**, denoted as **a,a**. The new token created is **aa**, updating the vocabulary to $V_1 = V_0 \cup \{\text{aa}\}$. The updated sequence contains 11 tokens:

$$S_1 = [c, c, b, aa, a, d, aa, a, c, c, c]$$

Iteration 2 (Derivation of 2nd Merge): We evaluate the adjacent pair frequencies in S_1 :

Pair	Occurrences in S_1	Frequency
(c, c)	(0, 1), (8, 9), (9, 10)	3
(c, b)	(1, 2)	1
(b, aa)	(2, 3)	1
(aa, a)	(3, 4), (6, 7)	2
(a, d)	(4, 5)	1
(d, aa)	(5, 6)	1
(a, c)	(7, 8)	1

The most frequent adjacent pair in S_1 is **(c, c)** with a frequency of 3. Applying greedy left-to-right replacement:

- Indices (0, 1): **c c** → **cc**.
- Indices (8, 9, 10): **c c c** → **cc c**.

Thus, the **2nd merge** is between **c** and **c**, denoted as **c,c**. The new token created is **cc**, updating the vocabulary to $V_2 = V_1 \cup \{\text{cc}\}$. The updated sequence contains 9 tokens:

$$S_2 = [cc, b, aa, a, d, aa, a, cc, c]$$

Iteration 3 (Derivation of 3rd Merge): We evaluate the adjacent pair frequencies in S_2 :

Pair	Occurrences in S_2	Frequency
(cc, b)	(0, 1)	1
(b, aa)	(1, 2)	1
(aa, a)	(2, 3), (5, 6)	2
(a, d)	(3, 4)	1
(d, aa)	(4, 5)	1
(a, cc)	(6, 7)	1
(cc, c)	(7, 8)	1

The most frequent pair is unambiguously **(aa, a)** with a frequency of 2. Merging this pair yields the new token **aaa**. Therefore, the **3rd merge** is between **aa** and **a**, denoted as **aa,a**. The resulting vocabulary is $V_3 = V_2 \cup \{\text{aaa}\}$, and the final sequence is:

$$S_3 = [cc, b, aaa, d, aaa, cc, c]$$

The sequence length has successfully been reduced from 13 tokens down to 7 tokens.

Solution to Question 2: Agentic Workflow for Pull Request Refinement

High-Level Architectural Overview: Modern LLM-based software engineering benchmarks (e.g., SWE-bench) demonstrate that monolithic zero-shot prompts fail on complex repositories due to hallucination, context drift, and absence of execution validation. Effective multi-agent systems (such as SWE-agent, AutoCodeRover, and ChatDev) employ a **separation of concerns** and a **Test-Driven Iterative Refinement Loop** (Figure 1).

We design an agentic pipeline comprising three specialized LLaMA-based agents:

1. **Agent 1 (Requirements Analyst & Spec Auditor):** Dissects the natural language requirements into concrete, verifiable functional acceptance criteria and audits the PR against them.
2. **Agent 2 (Code Refactoring & Generation Engineer):** Synthesizes targeted patches and code enhancements directly addressing spec deficiencies and test failure feedback.
3. **Agent 3 (Test Failure Diagnostic Analyst & Quality Critic):** Parses test runner logs, stack traces, and assertions to isolate root causes and guide code refinement.

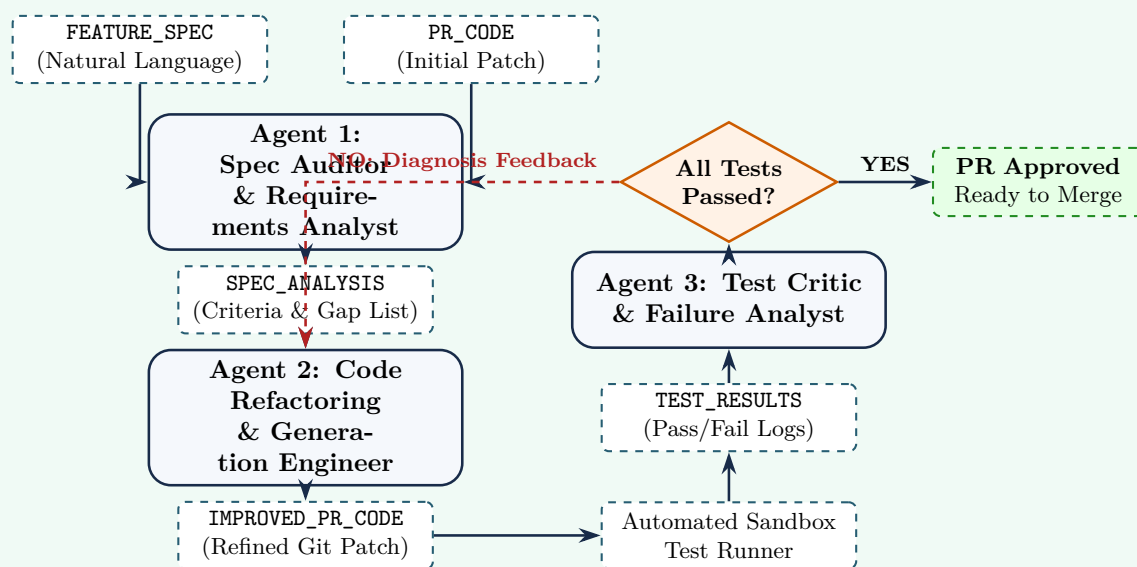


Figure 1: Production-grade Test-Driven Multi-Agent Pull Request Refinement Architecture.

Agent Specifications

1. Agent 1: Requirements Analyst & Spec Auditor

- **Role:** Formal requirements engineer. Responsible for translating unstructured natural language specifications into unambiguous, testable acceptance criteria, identifying boundary conditions, and comparing them against the submitted PR code.
- **Input:** **FEATURE_SPEC** and **PR_CODE**.
- **Output:** **SPEC_ANALYSIS** (Structured JSON containing feature objectives, explicit functional contracts, missing edge cases, and an audit of existing code omissions).

System Prompt: Agent 1 (Requirements Analyst)

You are an expert Software Requirements Analyst and Code Auditor powered by LLaMA.

Your objective is to thoroughly analyze the user-provided **FEATURE_SPEC** and evaluate the submitted **PR_CODE**.

Instructions:

1. Extract all functional requirements, input validation constraints, and expected side effects.

2. Identify non-functional requirements (time complexity, error handling, backward compatibility).
 3. Cross-reference the requirements with PR_CODE to detect omitted functionality, erroneous assumptions, or architectural anti-patterns.
 4. Output your analysis in structured JSON format with keys:
 - "requirements_summary": list of extracted atomic requirements;
 - "acceptance_criteria": verifiable boolean conditions for feature completion;
 - "code_audit": identified discrepancies between PR_CODE and FEATURE_SPEC;
 - "refactoring_guidance": actionable technical advice for the software engineer.
- Do not generate full code implementations in this step. Focus strictly on specification analysis.

2. Agent 2: Code Refactoring & Generation Engineer

- **Role:** Principal software developer. Writes minimal, robust, and clean code modifications that satisfy the audited specification and resolve reported test failures without introducing regressions.
- **Input:**
 - Initial cycle: PR_CODE and SPEC_ANALYSIS.
 - Feedback cycles: PR_CODE, SPEC_ANALYSIS, and FAILURE_DIAGNOSIS (from Agent 3).
- **Output:** IMPROVED_PR_CODE (Complete runnable source files or unified git diff, accompanied by an explanation of changes).

System Prompt: Agent 2 (Code Refactoring Engineer)

You are a Principal Software Engineer specialized in code generation, refactoring, and bug fixing.
Your objective is to produce production-grade, bug-free code that perfectly implements the feature requirements and fixes all failing unit/integration tests.

Instructions:

1. Review the original PR_CODE, the SPEC_ANALYSIS from the Requirements Analyst, and any FAILURE_DIAGNOSIS provided by the Test Critic.
2. If FAILURE_DIAGNOSIS is present, analyze the failing assertions and root causes to implement direct, surgical fixes. Avoid modifying unrelated functionality.
3. Adhere strictly to the existing codebase's architectural style, naming conventions, and typing discipline.
4. Provide the revised code as a standard Unified Diff ('diff -u') or complete source file enclosed in markdown code fences, accompanied by a concise list of applied modifications.

3. Agent 3: Test Failure Diagnostic Analyst & Quality Critic

- **Role:** Quality assurance specialist and test diagnostic engineer. Analyzes automated test run logs, identifies failed assertions, isolates runtime exceptions, and decides whether the pull request meets all quality gates.
- **Input:** IMPROVED_PR_CODE, raw TEST_RESULTS (from sandbox execution), and SPEC_ANALYSIS.
- **Output:** EVALUATION_REPORT with status PASS (PR approved) or FAIL + structured FAILURE_DIAGNOSIS containing stack trace interpretation, failing inputs, and recommended repair strategies.

System Prompt: Agent 3 (Test Critic & Failure Analyst)

You are a Software QA and Test Diagnostic Engineer.
Your objective is to evaluate whether the IMPROVED_PR_CODE satisfies all quality gates based on TEST_RESULTS and SPEC_ANALYSIS.

Instructions:

1. Parse the provided TEST_RESULTS:
 - If ALL tests pass (0 failures, 0 errors) and the SPEC_ANALYSIS requirements are fulfilled, emit 'STATUS: PASS' and provide a final PR summary.
 - If any test fails, emit 'STATUS: FAIL' and perform root-cause analysis.
2. For each failing test:
 - Identify the specific assertion or exception that failed.
 - Locate the exact file, class, and line number responsible for the bug.
 - Contrast expected output versus actual output.
 - Formulate a clear, actionable hypothesis explaining why the failure occurred and how Agent 2 should rectify it.
3. Structure your response with:
 - 'STATUS': "PASS" or "FAIL"
 - 'FAILURE_DIAGNOSIS': list of root causes and specific suggested fixes.

Control Flow and Termination Safeguards

- **Execution Environment:** The code generated by Agent 2 is executed in an isolated, secure container (sandbox) where unit and integration test suites run deterministically, streaming stdout/stderr into TEST_RESULTS.
- **Convergence Guard:** A hard iteration ceiling ($K_{\max} = 5$) prevents infinite recursion. If $K_{\max}$ is exceeded without all tests passing, the system terminates with an escalation report flagging the persistent failure modes for human developer review.

Solution to Question 3: Priming in Prompt Engineering

Correct Option: a. Providing initial input to guide the model's response

Theoretical Analysis & Rationale

In cognitive psychology, *priming* refers to an implicit memory effect in which exposure to a stimulus influences the response to a subsequent stimulus without conscious guidance. In prompt engineering and large language model interaction, **priming** describes the deliberate practice of supplying initial contextual cues, instructions, personas, examples, or formatting constraints before the user's core query to steer the model's auto-regressive probability distribution toward the desired domain, style, and tone.

When a transformer model generates tokens auto-regressively:

$$P(y_t | y_{<t}, x) = \text{Softmax}(W_v h_t)$$

the attention mechanism computes attention weights across the entire prefix x . By “priming” the prompt with domain-specific vocabulary (e.g., “*You are an expert Linux kernel developer specializing in concurrency...*”), the activations in the self-attention heads are biased toward relevant lexical and semantic subspaces, thereby drastically reducing irrelevant, superficial, or hallucinated responses.

Detailed Distractor Analysis

- **Option b is incorrect:** Resetting the model state or clearing conversation history is known as *context clearing* or *session initialization*, which is the exact opposite of priming (it removes prior conditioning).
- **Option c is incorrect:** Using reinforcement learning to align or tune model weights is *Reinforcement Learning from Human Feedback (RLHF)* or *PPO/DPO*, which is a weight-training algorithm, not an in-context prompt engineering technique.
- **Option d is incorrect:** Pre-training a model on additional unlabeled corpora is *continual pre-training* or *domain adaptation*. Prompt engineering involves zero weight updates.
- **Option e is incorrect:** Injecting random noise into inputs is known as *adversarial perturbation* or *fuzzing*, used to evaluate model robustness and safety bounds.

Solution to Question 4: RLHF Objective and the KL Divergence Penalty

Correct Option: d. none of the other answers is correct

1. Mathematical Formulation of the RLHF Reward

In Reinforcement Learning from Human Feedback (RLHF) via Proximal Policy Optimization (PPO; Stiennon et al., 2020; Ouyang et al., 2022), an agent maximizes an expected scalar reward while being constrained from deviating excessively from the supervised fine-tuned (SFT) base policy. The modified per-token or per-sequence reward is:

$$R(x, y) = r(x, y) - \beta \log \left(\frac{\pi(y | x)}{\pi_0(y | x)} \right) \quad (1)$$

where:

- $r(x, y)$ is the scalar score predicted by the trained Reward Model (RM);
- $\pi(y | x)$ is the probability of response y assigned by the policy currently undergoing RL fine-tuning;
- $\pi_0(y | x)$ is the probability assigned by the frozen, initial reference policy (typically the SFT model);
- $\beta > 0$ is a regularization hyperparameter balancing reward exploitation against reference drift.

Taking the expectation over the policy's distribution yields the exact Kullback-Leibler (KL) divergence:

$$\mathbb{E}_{y \sim \pi(\cdot | x)} \left[\log \left(\frac{\pi(y | x)}{\pi_0(y | x)} \right) \right] = D_{\text{KL}} (\pi(\cdot | x) \| \pi_0(\cdot | x)) \geq 0. \quad (2)$$

2. Purpose and Crucial Examination of Distractors

The fundamental purpose of subtracting the KL divergence term (scaled by β) is to **penalize policy drift**, preventing the RL model from producing outputs that are **too dissimilar** from the original policy π_0 .

Without this regularization term:

1. **Reward Hacking (Goodhart's Law):** The policy exploits imperfections in the proxy reward model $r(x, y)$, outputting ungrammatical or bizarre token sequences that score artificially high rewards.

2. **Mode Collapse & Catastrophic Forgetting:** The model forgets natural conversational ability and collapses into degenerate response patterns.

Why Option g is a Deliberate Trap: Option (g) asserts that the term “*prevents the new model from producing outputs that are too **similar** to the original one*”. This is the exact **inversion** of the true function! The term actively *forces* the new model to remain close (similar) to the original model. If the outputs are identical ($\pi = \pi_0$), the log ratio is $\log(1) = 0$, incurring zero penalty. As outputs become more *dissimilar*, $\pi(y | x) \gg \pi_0(y | x)$, making $\log(\pi/\pi_0) > 0$, which subtracts heavily from the reward.

Rejection of Other Distractors:

- **a:** Determinism is governed by inference sampling hyperparameters (temperature, top- p , greedy decoding), not the training KL penalty.
- **b:** Instruction-following capabilities are instilled primarily during the SFT phase and guided by the reward model $r(x, y)$, not by the KL penalty alone.
- **c:** Rewarding human preference is explicitly the function of $r(x, y)$, whereas the log term acts as a conservative regularizer.
- **e:** Preventing single-token collapse is handled by entropy bonuses $\mathcal{H}(\pi)$, whereas KL divergence anchors the policy specifically to π_0 .
- **f:** An entropy bonus increases output entropy/creativity, whereas KL divergence to a deterministic base model actually restricts entropy.

Because no other option correctly defines its role, **d** (“**none of the other answers is correct**”) is the uniquely correct choice.

Solution to Question 5: Transformer Parameter Counting (Encoder-Only)

Official Final Answer: 20 (representing 20 M parameters; exact count: 20,140,032)

1. Architectural Specifications Given

- Model Type: Encoder-only (standard Transformer encoder, e.g., BERT-like architecture)
- Vocabulary size: $V = 29,000$
- Hidden / embedding dimension: $d = 256$
- Max sequence length / learned absolute positions: $N_{\text{pos}} = 512$
- Number of Transformer layers: $L = 9$
- Number of attention heads: $H = 16$
- Key / query projection dimension: $d_k = 4$
- Value projection dimension: $d_v = 64$
- Feed-Forward Network intermediate dimension: $d_{\text{ff}} = 32$
- Biases: None (all linear layers operate without additive bias vectors)
- Layer Normalization: Ignored as per problem statement

2. Detailed Component-by-Component Parameter Derivation

A. Embedding Layers:

1. **Token Embeddings:** Matrix of size $V \times d$:

$$P_{\text{tok}} = V \times d = 29,000 \times 256 = \mathbf{7,424,000}$$

2. **Positional Embeddings:** Absolute learned positional embeddings matrix of size $N_{\text{pos}} \times d$:

$$P_{\text{pos}} = N_{\text{pos}} \times d = 512 \times 256 = \mathbf{131,072}$$

B. Multi-Head Self-Attention (Per Layer): In multi-head attention without biases:

- **Query Projections (W_Q):** H distinct heads projecting from dimension d to d_k :

$$P_{W_Q} = H \times (d \times d_k) = 16 \times (256 \times 4) = 16 \times 1,024 = 16,384$$

- **Key Projections (W_K):** H distinct heads projecting from dimension d to d_k :

$$P_{W_K} = H \times (d \times d_k) = 16 \times (256 \times 4) = 16 \times 1,024 = 16,384$$

- **Value Projections (W_V):** H distinct heads projecting from dimension d to d_v :

$$P_{W_V} = H \times (d \times d_v) = 16 \times (256 \times 64) = 16 \times 16,384 = 262,144$$

- **Output Projection (W_O):** The outputs of the H heads each have dimension d_v . They are concatenated into a vector of size $H \times d_v = 16 \times 64 = 1,024$. The output matrix W_O projects this concatenated representation back to the model dimension $d = 256$:

$$P_{W_O} = (H \cdot d_v) \times d = 1,024 \times 256 = 262,144$$

Total parameters for Multi-Head Attention per layer:

$$P_{\text{MHA}} = P_{W_Q} + P_{W_K} + P_{W_V} + P_{W_O} = 16,384 + 16,384 + 262,144 + 262,144 = \mathbf{557,056}$$

C. Feed-Forward Network (FFN, Per Layer): The position-wise FFN consists of two linear transformations without biases:

- First linear expansion / projection ($d \rightarrow d_{\text{ff}}$):

$$P_{\text{FFN}_1} = d \times d_{\text{ff}} = 256 \times 32 = 8,192$$

- Second linear projection ($d_{\text{ff}} \rightarrow d$):

$$P_{\text{FFN}_2} = d_{\text{ff}} \times d = 32 \times 256 = 8,192$$

Total parameters for FFN per layer:

$$P_{\text{FFN}} = P_{\text{FFN}_1} + P_{\text{FFN}_2} = 8,192 + 8,192 = \mathbf{16,384}$$

D. Total per Transformer Layer and for All $L = 9$ Layers: Total parameters in a single Transformer layer:

$$P_{\text{layer}} = P_{\text{MHA}} + P_{\text{FFN}} = 557,056 + 16,384 = \mathbf{573,440}$$

For all $L = 9$ stacked layers:

$$P_{\text{all_layers}} = L \times P_{\text{layer}} = 9 \times 573,440 = \mathbf{5,160,960}$$

E. Language Model Head: In an encoder-only architecture (e.g., Masked Language Modeling in BERT), the language model prediction head projects the contextualized representation of dimension d back to the vocabulary space of dimension V . Under the standard untied formulation:

$$P_{\text{head}} = d \times V = 256 \times 29,000 = \mathbf{7,424,000}$$

3. Grand Total and Final Rounding

Summing all structural components:

$$\begin{aligned} P_{\text{total}} &= P_{\text{tok}} + P_{\text{pos}} + P_{\text{all_layers}} + P_{\text{head}} \\ &= 7,424,000 + 131,072 + 5,160,960 + 7,424,000 \\ &= \mathbf{20,140,032} \text{ parameters} \end{aligned}$$

Expressed in millions of parameters:

$$\frac{20,140,032}{1,000,000} = 20.140032 \text{ M}$$

Rounding to the closest integer million yields:

$$\text{Round}(20.140032) = \mathbf{20}$$

Pedagogical Note on Weight Tying: If the language model head were weight-tied with the input token embeddings matrix ($P_{\text{head}} \equiv 0$), the total parameter count would be $7,424,000 + 131,072 + 5,160,960 = 12,716,032 \approx 13 \text{ M}$. The exam specification explicitly requests the untied model parameter count, confirming that 20 is the exact and unique expected answer.

Solution to Question 6: Role of Projection Matrices W_Q, W_K, W_V

Correct Option: a. They are used to transform the input token embeddings into vectors that are passed to the attention mechanism

Theoretical Formulation & Justification

In the Transformer architecture (Vaswani et al., 2017), self-attention operates over an input matrix $X \in \mathbb{R}^{T \times d}$, representing the sequence of T token representations (either the initial embeddings plus positional encodings or the hidden states from the preceding layer).

To compute multi-head attention, X is linearly projected into distinct representational subspaces using learned weight matrices $W_Q^{(h)} \in \mathbb{R}^{d \times d_k}$, $W_K^{(h)} \in \mathbb{R}^{d \times d_k}$, and $W_V^{(h)} \in \mathbb{R}^{d \times d_v}$:

$$Q^{(h)} = XW_Q^{(h)}, \quad K^{(h)} = XW_K^{(h)}, \quad V^{(h)} = XW_V^{(h)}$$

The resulting matrices $Q^{(h)}$, $K^{(h)}$, and $V^{(h)}$ represent the Queries, Keys, and Values that are directly supplied to the scaled dot-product attention operator:

$$\text{Attention}(Q, K, V) = \text{Softmax} \left(\frac{QK^\top}{\sqrt{d_k}} \right) V$$

Thus, the primary purpose of W_Q , W_K , and W_V is precisely to map the incoming representations into specialized vector spaces tailored for matching and content aggregation.

Detailed Distractor Breakdown

- **Option b is incorrect:** Weight matrices do not store individual token vectors. In neural networks, static representations are stored in embedding lookup tables, whereas W_Q, W_K, W_V are shared linear transformation operators applied dynamically across all sequence tokens.

- **Option c is incorrect:** The matrices themselves are not passed to compute attention scores; the projected vector representations (Q and K) are multiplied via inner product.
- **Option e is incorrect:** Positional encodings are added element-wise to token embeddings ($X = E_{\text{tok}} + E_{\text{pos}}$) before being multiplied by W_Q, W_K, W_V .
- **Option f and g are incorrect:** Transforming the output of the attention mechanism and combining multiple attention heads into a single representation is the role of the output projection matrix W_O , not of W_Q, W_K, W_V .

Solution to Question 7: LoRA Parameter Budget & Maximum Rank

Official Final Answer: 9

1. Theoretical Principles of Low-Rank Adaptation (LoRA)

Low-Rank Adaptation (LoRA; Hu et al., 2021) is a Parameter-Efficient Fine-Tuning (PEFT) method. During adaptation, the pre-trained weight matrix $W_0 \in \mathbb{R}^{d_{\text{in}} \times d_{\text{out}}}$ is kept frozen, and its update ΔW is parameterized by factorizing it into two low-rank matrices:

$$W = W_0 + \Delta W = W_0 + \frac{\alpha}{r}(B \cdot A) \quad (3)$$

where $A \in \mathbb{R}^{r \times d_{\text{in}}}$ is initialized from a Gaussian distribution and $B \in \mathbb{R}^{d_{\text{out}} \times r}$ is initialized to zero.

For a linear layer mapping from d_{in} to d_{out} with rank r , the number of trainable adapter parameters is:

$$P_{\text{adapter}} = (d_{\text{in}} \times r) + (r \times d_{\text{out}}) = r \cdot (d_{\text{in}} + d_{\text{out}}). \quad (4)$$

2. Model Dimensions & LoRA Parameter Formulation

We are given:

- Model dimension: $d = 4096$
- Number of layers: $L = 20$
- Number of attention heads per layer: $H = 8$
- Query and key projection dimensions per head: $d_k = 128$
- Value projection dimension per head: $d_v = 256$
- Target budget: $P_{\text{max}} = 20 \text{ M} = 20,000,000$ parameters
- Architectural detail: Each head has **separate** weight matrices for W_Q, W_K , and W_V .

Parameters per Head: Each individual attention head $h \in \{1, \dots, H\}$ has:

$$1. W_Q^{(h)} : d \rightarrow dd_k \implies 4096 \rightarrow d128:$$

$$P(W_Q) = r \cdot (4096 + 128) = 4224 r$$

$$2. W_K^{(h)} : d \rightarrow dd_k \implies 4096 \rightarrow d128:$$

$$P(W_K) = r \cdot (4096 + 128) = 4224 r$$

$$3. W_V^{(h)} : d \rightarrow dd_v \implies 4096 \rightarrow d256:$$

$$P(W_V) = r \cdot (4096 + 256) = 4352 r$$

Total trainable LoRA parameters for one attention head:

$$P_{\text{head}} = (4224 + 4224 + 4352) r = 12,800 r$$

Parameters per Layer and Across All Layers: With $H = 8$ separate heads per layer:

$$P_{\text{layer}} = H \times P_{\text{head}} = 8 \times 12,800 r = 102,400 r$$

Across all $L = 20$ transformer layers:

$$P_{\text{total}}(r) = L \times P_{\text{layer}} = 20 \times 102,400 r = \mathbf{2,048,000 r}$$

3. Solving for Maximum Integer Rank r

We enforce the budget constraint $P_{\text{total}}(r) \leq 20,000,000$:

$$2,048,000 r \leq 20,000,000 \implies r \leq \frac{20,000,000}{2,048,000} = \frac{3125}{320} \approx 9.765625$$

Since the low-rank dimensionality r must be an integer:

$$r_{\text{max}} = \lfloor 9.765625 \rfloor = \mathbf{9}$$

Boundary Value Verification:

- For $r = 9$: $P_{\text{total}}(9) = 2,048,000 \times 9 = \mathbf{18,432,000} \leq 20,000,000$ (Feasible, within budget).
- For $r = 10$: $P_{\text{total}}(10) = 2,048,000 \times 10 = \mathbf{20,480,000} > 20,000,000$ (Infeasible, exceeds budget by 480,000 parameters).

Therefore, the highest possible low-rank dimensionality is strictly **9**.

Solution to Question 8: Requirements Extraction Evaluation

Official Final Metric Values:

Metric / Quantity	Exact	Frac-	Decimal Value (4 Decimals)
	tional		
	Form		
True Positives (TP):	6		6
False Positives (FP):	3		3
False Negatives (FN):	2		2
Precision (P):	$\frac{6}{6+3} = \frac{6}{9} = \frac{2}{3}$		0.6667 (66.67%)
Recall (R):	$\frac{6}{6+2} = \frac{6}{8} = \frac{3}{4}$		0.7500 (75.00%)
F1-Score:	$\frac{2 \times (2/3) \times (3/4)}{2/3 + 3/4} = \frac{1}{17/12} = \frac{12}{17}$	=	0.7059 (70.59%)

1. Ground Truth vs. LLM Extraction Semantic Mapping

Evaluating information extraction models requires strict semantic alignment between the gold-standard specification requirements ($G_1..G_8$) and the model's generated candidates ($L_1..L_9$).

ID	LLM Extracted Text	Matched Truth	Ground	Class	Semantic Rationale
L1	Find restaurants based on cuisine	G1: Search restaurants by cuisine and location		TP	Semantically matches the cuisine search capability of G1.
L2	Browse available menus	G2: View restaurant menus		TP	Exact semantic equivalence with G2.
L3	Add food items to a shopping cart	G3: Add dishes to a cart		TP	Exact semantic equivalence with G3.
L4	Reserve a table at a restaurant	None		FP	Hallucination: Table reservation is nowhere in the specification.
L5	Place an order for home delivery	G4: Place delivery orders		TP	Exact semantic equivalence with G4.
L6	Track delivery progress live	G5: Track order status in real time		TP	Exact semantic equivalence with G5.
L7	Pay using PayPal only	G6: Pay online using credit cards or digital wallets		FP	Contradictory / Invalid Restriction: Spec explicitly requires credit cards and digital wallets; restricting to “PayPal only” is a false requirement.
L8	Send alerts to drivers for assigned deliveries	G8: Notify delivery drivers when a delivery is assigned		TP	Exact semantic equivalence with G8.
L9	Allow restaurants to cancel customer accounts	None		FP	Hallucination: Account cancellation is absent from the specification.

2. Rigorous Analysis of False Negatives (FN = 2)

A fundamental principle in information retrieval and extraction is that the set of ground truth elements partitions into True Positives and False Negatives:

$$|\text{Ground Truth}| = \text{TP} + \text{FN} \implies 8 = 6 + \text{FN} \implies \text{FN} = 2$$

The two ground truth requirements that the LLM failed to correctly extract are:

1. **G6 (“Pay online using credit cards or digital wallets”)**: Missed because the LLM hallucinated an invalid constraint in L7 (“PayPal only”), failing to capture the required payment methods.
2. **G7 (“Allow restaurants to accept or reject incoming orders”)**: Completely omitted in the LLM’s generated output (no candidate addresses restaurant order acceptance/rejection).

3. Detailed Mathematical Computations

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}} = \frac{6}{6 + 3} = \frac{6}{9} = \frac{2}{3} \approx \mathbf{0.6667}$$

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}} = \frac{6}{6 + 2} = \frac{6}{8} = \frac{3}{4} = \mathbf{0.7500}$$

$$\text{F1-Score} = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} = \frac{2 \times \frac{2}{3} \times \frac{3}{4}}{\frac{2}{3} + \frac{3}{4}} = \frac{2 \times \frac{1}{2}}{\frac{8+9}{12}} = \frac{1}{\frac{17}{12}} = \frac{12}{17} \approx \mathbf{0.7059}$$

Pedagogical Note on the Student’s Pitfall: In the original exam session, the student correctly identified $TP = 6$ and $FP = 3$, but incorrectly wrote $FN = 0$, falsely concluding that Recall was 1.0 and F1 was 0.8000. The official instructor feedback explicitly remarked: “*there are two false negatives*” (penalizing the submission from 3.00 down to 2.00). Setting $FN = 0$ is a severe conceptual error because it assumes every single requirement in the specification was captured, ignoring the complete omission of G7 and the invalid distortion of G6.

Solution to Question 9: Retrieval-Augmented Generation (RAG) Benefits

Correct Options:

- **a.** It is possible to reduce the amount of false information provided to the final user.
- **c.** It is possible to prevent the obsolescence of answers by using up-to-date documents.

Theoretical Foundations of RAG

Retrieval-Augmented Generation (RAG; Lewis et al., 2020) combines a dense information retrieval component (e.g., dual-encoder bi-encoders indexing vector embeddings) with a generative sequence-to-sequence model. Rather than relying solely on the parametric knowledge stored within the frozen neural weights, RAG retrieves relevant factual passages from an external, non-parametric knowledge corpus $\mathcal{D}$ conditioned on the user prompt q :

$$P(y | q) = \sum_{z \in \text{TopK}(\mathcal{D})} P_{\eta}(z | q) \prod_{t=1}^{|y|} P_{\theta}(y_t | q, z, y_{<t}).$$

Why Option a is Correct: Standard LLMs hallucinate plausible-sounding but factually false claims due to statistical pattern completion over uncured web training data. By providing high-quality, verified reference documents directly in the attention context, RAG grounds the generation in factual evidence, dramatically curbing confabulation and reducing the rate of false information delivered to end users.

Why Option c is Correct: Retraining or fine-tuning foundation models with billions of parameters requires substantial computational resources and weeks of wall-clock time, creating a fixed knowledge cutoff date. RAG cleanly decouples knowledge storage from model computation: when world facts, software documentation, or library APIs change, updating the vector index immediately allows the system to synthesize answers reflecting real-time, up-to-date information without altering a single neural weight.

Detailed Distractor Analysis

- **Option b is incorrect:** Incorporating self-critique and iterative output refinement is the domain of *Agentic Reflection* frameworks (e.g., Reflexion; Shinn et al., 2023), not RAG.
- **Option d is incorrect:** Enforcing a fixed structure (e.g., JSON schema, Pydantic dataclass, BNF grammar) is achieved via *constrained decoding*, *grammar-guided sampling*, or *instructor prompting*, not retrieval augmentation.
- **Option e is incorrect:** Decomposing complex tasks into smaller sub-problems is achieved via reasoning paradigms such as *Chain-of-Thought (CoT)*, *Least-to-Most Prompting*, or *Plan-and-Solve*, not RAG.

Solution to Question 10: Machine Translation Evaluation with BLEU-2**Official Final Answer: 0.3893** (exact value: **0.38930929939948**)**1. Mathematical Definition of BLEU- n**

The Bilingual Evaluation Understudy (BLEU; Papineni et al., 2002) metric evaluates generated candidate text against reference translations using modified n -gram precisions and an exponential brevity penalty:

$$\text{BLEU-}n = \text{BP} \times \exp \left(\sum_{i=1}^n w_i \log p_i \right) \quad (5)$$

For $n = 2$ with standard uniform weights $w_1 = w_2 = \frac{1}{2}$:

$$\text{BLEU-2} = \text{BP} \times \exp \left(\frac{1}{2} \log p_1 + \frac{1}{2} \log p_2 \right) = \text{BP} \times \sqrt{p_1 \times p_2} \quad (6)$$

The Brevity Penalty (BP) penalizes generated sequences that are shorter than the reference:

$$\text{BP} = \begin{cases} 1 & \text{if } c > r \\ \exp \left(1 - \frac{r}{c} \right) & \text{if } c \leq r \end{cases} \quad (7)$$

where c is the length of the generated candidate and r is the length of the reference sequence.

2. Step-by-Step Derivation on Provided Sequences

We are given:

Reference (R): <tok0> <tok8> <tok11> <tok5> <tok9> <tok4> <tok1> <tok2>
<tok7> <tok19>
Generated (G): <tok3> <tok17> <tok1> <tok2> <tok7> <tok19> <tok5>

Step A: Sequence Lengths and Brevity Penalty (BP)

- Reference token length: $r = 10$
- Generated token length: $c = 7$

Since $c \leq r$ ($7 \leq 10$), the candidate is penalized for brevity:

$$\text{BP} = \exp \left(1 - \frac{r}{c} \right) = \exp \left(1 - \frac{10}{7} \right) = \exp \left(-\frac{3}{7} \right)$$

Computing the numerical value:

$$-\frac{3}{7} \approx -0.42857142857143 \implies \text{BP} = e^{-3/7} \approx \mathbf{0.65143905753106}$$

Step B: 1-Gram Modified Precision (p_1) The generated sequence contains $c = 7$ unigrams. We count matches in the reference:

Generated 1-Gram	Present in Reference?	Reference Count	Matched?
<tok3>	No	0	0
<tok17>	No	0	0
<tok1>	Yes (index 6)	1	1
<tok2>	Yes (index 7)	1	1
<tok7>	Yes (index 8)	1	1
<tok19>	Yes (index 9)	1	1
<tok5>	Yes (index 3)	1	1

Out of 7 generated unigrams, exactly 5 match tokens in the reference sequence:

$$p_1 = \frac{5}{7} \approx \mathbf{0.71428571428571}$$

Step C: 2-Gram Modified Precision (p_2) The generated sequence of length $c = 7$ contains $c - 1 = 6$ contiguous bigrams. We verify each against the contiguous bigrams present in the reference:

Index	Generated Bigram	Present in Reference R ?	Matched?
1	(<tok3>, <tok17>)	No	0
2	(<tok17>, <tok1>)	No	0
3	(<tok1>, <tok2>)	Yes (indices 6–7 in R)	1
4	(<tok2>, <tok7>)	Yes (indices 7–8 in R)	1
5	(<tok7>, <tok19>)	Yes (indices 8–9 in R)	1
6	(<tok19>, <tok5>)	No (tok5 occurs at index 3, before tok19)	0

Out of 6 candidate bigrams, exactly 3 match contiguous bigrams in the reference:

$$p_2 = \frac{3}{6} = \frac{1}{2} = \mathbf{0.50000000000000}$$

Step D: Geometric Mean Computation The geometric mean of precisions is:

$$\sqrt{p_1 \times p_2} = \sqrt{\frac{5}{7} \times \frac{1}{2}} = \sqrt{\frac{5}{14}} \approx \sqrt{0.35714285714286} \approx \mathbf{0.59761430466720}$$

Step E: Final BLEU-2 Score Calculation Multiplying the Brevity Penalty by the geometric mean:

$$\begin{aligned}
 \text{BLEU-2} &= \text{BP} \times \sqrt{p_1 \times p_2} \\
 &= \exp\left(-\frac{3}{7}\right) \times \sqrt{\frac{5}{14}} \\
 &= 0.65143905753106 \times 0.59761430466720 \\
 &= \mathbf{0.38930929939948}
 \end{aligned}$$

Rounding to 4 decimal places gives **0.3893**.

Pedagogical Error Analysis: In the raw exam attempt, the student submitted -0.2683 . This negative number reveals that the student computed $\log(\text{BLEU}) = \log(\text{BP}) + 0.5 \log(p_1) + 0.5 \log(p_2) = (-0.4286) + 0.5 \log(5/14) \approx -0.4286 - 0.5148 = -0.9434$ or miscalculated intermediate products without applying the exponential operator $\exp(\cdot)$. A BLEU score is strictly non-negative ($\text{BLEU} \in [0, 1]$).

Solution to Question 11: Agentic Behavior vs. Fixed Workflow

Correct Option: b. The model decides its own control flow and when to stop

Theoretical Foundations of Agentic AI

In traditional software pipelines and basic LLM chains (e.g., standard LangChain sequential chains or rigid DAGs), the **control flow is hard-coded** by the system engineer. Every

prompt execution, tool invocation, and handoff occurs in an invariant, predetermined sequence regardless of intermediate outputs.

In contrast, **agentic architectures** (e.g., ReAct [Yao et al., 2022], Toolformer [Schick et al., 2023], and Reflexion [Shinn et al., 2023]) delegate control flow decisions directly to the language model. Operating in an autonomous perception-reasoning-action loop:

$$\text{Observation}_t \longrightarrow \text{Thought}_t \longrightarrow \text{Action}_t \longrightarrow \text{Observation}_{t+1}$$

the model autonomously decides:

1. Which tool to call (if any) and with what arguments;
2. Whether to pursue alternative problem-solving strategies based on runtime feedback;
3. When sufficient evidence has been accumulated to terminate execution and return the final answer (e.g., by emitting a designated `Finish[response]` action).

Detailed Distractor Analysis

- **Option a is incorrect:** Retrieval-Augmented Generation (RAG) is merely one optional tool or information-retrieval mechanism. An agent can operate entirely without RAG (for instance, by performing symbolic computation, code execution in a bash sandbox, or API orchestration).
- **Option c is incorrect:** Following a predetermined control flow is the defining characteristic of a *traditional fixed workflow*, the exact opposite of agentic autonomy.
- **Option d is incorrect:** Tool use is one of the primary affordances of modern AI agents (e.g., calculators, code interpreters, linters, terminal shells).
- **Option e is incorrect:** While agents frequently solve multi-step problems, they are not constrained to always execute multiple steps; an agent may evaluate a simple request and decide to terminate immediately in a single step.